

PostgreSQL Database Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 2 Months

Level: Intermediate

Course page: <https://archerinfotech.in/courses/database-technologies/postgresql-training-in-pune>

Course Overview

PostgreSQL is the dominant relational database in Pune product engineering and modern fintech / SaaS — Persistent Systems, BMC Software, Bajaj Finserv (significant Postgres footprint alongside Oracle), Synechron, BMW TechWorks India, Mastercard Pune Tech Hub, Amagi, Fyllo, BharatPe Pune, Razorpay Pune, Pine Labs Pune, Drip Capital, Innovaccer Pune, Whatfix Pune all run their primary OLTP workloads on it. Archer Infotech's PostgreSQL training in Pune teaches the database as it is actually used in 2026 — Postgres 16+ as the production default (Postgres 17 released Sept 2024 is gaining adoption), modern SQL features (window functions, CTEs, lateral joins), JSONB at depth (the Postgres differentiator), full-text search, partitioning, plus the rich extension ecosystem (pgvector for AI / RAG, PostGIS for geo, TimescaleDB for time-series, Citus for distribution). Classroom in Kothrud, online live, and weekend batches available.

Why Learn PostgreSQL in 2026

Postgres has eclipsed MySQL as the default choice for new Pune product engineering — Indeed Pune lists 700+ active openings that explicitly require PostgreSQL, plus several thousand more where Postgres is listed alongside other databases. The biggest employers running Postgres at scale are Persistent Systems, BMC Software, Bajaj Finserv, BMW TechWorks India, Mastercard Pune Tech Hub, plus the Pune SaaS / fintech scene (Amagi, Fyllo, BharatPe Pune, Razorpay Pune, Pine Labs Pune, Drip Capital, Innovaccer Pune, Whatfix Pune). Postgres also powers Amazon RDS / Aurora PostgreSQL, Azure Database for PostgreSQL Flexible Server, Cloud SQL Postgres, and Supabase / Neon — managed cloud variants several Pune teams ship on.

What changed in 2026: Postgres 16 (released Sept 2023) shipped logical replication improvements, parallel queries enhancements, plus better JSON path matching. Postgres 17 (released Sept 2024) shipped vector indexing improvements, incremental backup, plus performance gains. The pgvector extension has become the default vector database for RAG / AI integration in Pune product engineering — most Pune AI features use Postgres + pgvector instead of separate vector databases. The serverless Postgres trend (Neon, Supabase, AWS Aurora Serverless v2) has matured for the cases where elastic scaling matters.

What this means for hiring: 2026 Pune Postgres JDs expect SQL fluency at the window-functions / CTE / lateral-join level, JSONB depth, indexing and EXPLAIN-plan literacy, transactions and isolation, plus basic logical replication. Senior roles add partitioning, the major extensions (pgvector for AI / RAG; PostGIS for geo; TimescaleDB for time-series), plus performance tuning at depth. Archer Infotech's curriculum is rebuilt around exactly these expectations.

- 700+ active Pune openings explicitly require PostgreSQL (May 2026)
- Eclipsed MySQL as default for new Pune product engineering
- Postgres 16+ — current production default; Postgres 17 gaining adoption
- JSONB + window functions + lateral joins — modern Postgres features
- pgvector for AI / RAG integration — the 2026 differentiator

Who This Course Is For

This course fits you if

- Working backend / full-stack developer wanting Postgres depth (the database your team probably runs)
- Engineering, BCS, MCA, or BSc-CS student targeting Pune product engineering / fintech / SaaS
- Working MySQL developer wanting to add Postgres for the wider modern-product hiring market
- Working Data Engineer wanting Postgres-as-warehouse / Postgres-on-AI depth
- Career restarter targeting Database Administrator / Backend / Analytics-Engineer roles
- Working AI Engineer wanting pgvector + RAG depth

This course is not the right fit if

- If your goal is BFSI Pune Capital Markets DBA premium — pick our Oracle Database course (BFSI runs more Oracle at the high end)
- If your goal is LAMP / WordPress / digital-agency work — pick our MySQL course (more institutional fit)
- If you want NoSQL / document-database — pick our MongoDB course
- If you cannot put in 6–8 hours per week of practice outside class
- If you have 3+ years of production Postgres DBA experience — talk to us about advanced specialisations

Detailed Curriculum

Module 1: PostgreSQL Foundations & the Server (Week 1)

What PostgreSQL is and how it is put together, before any query is written. The relational model as a way of removing duplication, the process architecture — postmaster, backend processes, the write-ahead log and the shared buffers — and the cluster, database, schema and search-path hierarchy that confuses newcomers more than anything else in Postgres.

Installation covers Linux, macOS, Windows, Docker and the managed options a Pune team is likely to be handed instead: Amazon RDS, Azure Flexible Server, Cloud SQL, Neon and Supabase. `psql` is set up properly — meta-commands, `\timing`, output formats and `.psqlrc` — alongside pgAdmin, DBeaver and TablePlus, and the week closes on server configuration, roles at first sight, and `pg_hba.conf`, which is where most first connection failures live.

- The relational model and where Postgres fits
- Process architecture — postmaster, backends, WAL, shared buffers
- Cluster, database, schema and the search path
- Install on Linux, macOS, Windows and Docker
- Managed Postgres — RDS, Azure, Cloud SQL, Neon, Supabase
- `psql` meta-commands, `\timing` and `.psqlrc`
- pgAdmin, DBeaver and TablePlus
- `postgresql.conf`, `pg_hba.conf` and first-connection failures

Module 2: Core SQL & the Postgres Type System (Week 1)

The statements of a working day, taught in the order the server evaluates them — FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT — because that order is what explains when an alias resolves and when it does not. Aggregates, DISTINCT ON (a Postgres-only shortcut worth knowing), CASE, and NULL with the three-valued logic that quietly corrupts reports.

The type system is where Postgres pulls ahead and it is treated as a design tool rather than a list. Numeric types and NUMERIC for money, text and the collation question, the date and time family with `timestamptz` versus `timestamp` explained through an actual time-zone bug, UUID, ENUM, ranges, arrays, INTERVAL, and user-defined domains and composite types.

- SELECT clauses in logical evaluation order
- Aggregates, GROUP BY, HAVING and FILTER
- DISTINCT ON — the Postgres shortcut
- NULL, three-valued logic and COALESCE
- Numeric types and NUMERIC for money

- text, varchar, collations and citext
- timestampz vs timestamp, and the bug that teaches it
- UUID, ENUM, ranges, arrays and INTERVAL
- Domains and composite types

Module 3: Constraints, Keys & Table Design (Week 2)

The database enforcing what the application forgot. Primary keys and the identity-column syntax that replaced `serial`, foreign keys with every `ON DELETE` and `ON UPDATE` action, `UNIQUE`, `NOT NULL`, `CHECK`, and the exclusion constraint — a Postgres speciality that makes overlapping bookings impossible at the storage layer rather than in a race-prone application check.

Deferrable constraints, partial unique indexes for soft-deleted rows, generated columns and sensible defaults follow. The module ends on the modelling decisions that are expensive to reverse: surrogate versus natural keys, UUID version 4 versus version 7 as a primary key and what each costs on an index, and how to represent one-to-many, many-to-many and hierarchical relationships.

- Identity columns, and why `serial` is legacy
- Foreign keys and `ON DELETE` / `ON UPDATE` actions
- `UNIQUE`, `NOT NULL`, `CHECK` and `DEFAULT`
- Exclusion constraints for non-overlapping ranges
- Deferrable constraints and when they help
- Partial unique indexes for soft deletes
- Generated columns
- Surrogate vs natural keys, UUIDv4 vs UUIDv7
- Modelling one-to-many, many-to-many and hierarchies

Module 4: Joins, Subqueries & LATERAL (Week 2)

Combining tables correctly, and the two failures that produce most wrong numbers in reports: the accidental cartesian product, and the outer join silently converted to an inner join by a `WHERE` clause on the nullable side. Both are made deliberately in class before they are corrected.

Subqueries are covered in every position — scalar, `IN` list, derived table — with the correlated and uncorrelated distinction made concrete by execution cost. `EXISTS` and `NOT EXISTS` are shown as the right way to express semi- and anti-joins, and `LATERAL` gets a section of its own: it is the clean answer to top-N-per-group, and it is one of the things a Postgres interview asks about specifically.

- Inner, left, right, full and cross joins
- The accidental cartesian product
- How `WHERE` turns an outer join into an inner join
- Scalar, `IN`-list and derived-table subqueries
- Correlated vs uncorrelated and what each costs
- `EXISTS` and `NOT EXISTS` for semi- and anti-joins
- `LATERAL` joins and top-N-per-group
- `UNION`, `INTERSECT` and `EXCEPT`
- Set-returning functions in `FROM`

Module 5: CTEs, Window Functions & Analytical SQL (Week 3)

The SQL that replaces a page of application code. Common table expressions for readable query structure, `MATERIALIZED` and `NOT MATERIALIZED` since Postgres 12 (the optimisation fence is no longer automatic, and knowing that is worth marks in an interview), recursive CTEs for org charts, category trees and graph walks, and writable CTEs that return the rows they just modified.

Window functions are then taught as the tool for the problems that used to need a self-join: running totals, per-group ranking, month-on-month change with `LAG` and `LEAD`, moving averages over an explicit frame, and percentile and distribution functions. `GROUPING SETS`, `CUBE` and `ROLLUP` close the module for multi-level reporting in a single pass.

- CTEs and readable query structure
- MATERIALIZED vs NOT MATERIALIZED since Postgres 12
- Recursive CTEs for trees and graph traversal
- Writable CTEs with RETURNING
- Window frames — PARTITION BY, ORDER BY, ROWS and RANGE
- ROW_NUMBER, RANK, DENSE_RANK and NTILE
- LAG, LEAD, FIRST_VALUE and LAST_VALUE
- Percentiles and ordered-set aggregates
- GROUPING SETS, CUBE and ROLLUP

Module 6: JSONB, Arrays & Semi-Structured Data (Week 3)

The Postgres differentiator, treated as an engineering choice rather than a feature demo. json versus jsonb and why the binary form is almost always right, the containment and existence operators, path extraction, SQL/JSON path queries, and the functions that build JSON documents directly out of a relational query — the pattern that lets an API return nested objects without an ORM assembling them.

Indexing semi-structured data is where the performance lives: GIN indexes with the default and jsonb_path_ops operator classes, expression indexes on a single extracted key, and the check that the index is actually being used. Arrays and their operators follow, and the module ends on the judgement call — when a jsonb column is genuinely right, and when it is a normalised table that somebody avoided designing.

- json vs jsonb and the cost of each
- Containment and existence operators — @>, ?, ?&
- Path extraction with ->, ->> and #>
- SQL/JSON path queries
- Building JSON from relational rows for APIs
- GIN indexes and jsonb_path_ops
- Expression indexes on extracted keys
- Array types, operators and unnest
- When jsonb is right, and when it is avoidance

Module 7: Index Types & Index Design (Week 4)

Postgres has more index types than any comparable database, and choosing between them is a real skill. B-tree as the default and the leftmost-prefix rule that decides whether a composite index is used at all; GIN for jsonb, arrays and full-text; GiST for ranges and geometry; BRIN for naturally ordered large tables where it costs almost nothing; and hash and SP-GiST for their narrow cases.

Design then follows the query rather than the column: composite ordering driven by selectivity and by the WHERE and ORDER BY shape, covering indexes with INCLUDE, partial indexes that index only the rows anyone queries, and expression indexes for lower (email) lookups. The module ends on the costs — write amplification, bloat, REINDEX CONCURRENTLY, and finding unused indexes in pg_stat_user_indexes.

- B-tree, and the leftmost-prefix rule
- GIN for jsonb, arrays and full-text
- GiST for ranges and geometry; BRIN for ordered tables
- Hash and SP-GiST — the narrow cases
- Composite index ordering and selectivity
- Covering indexes with INCLUDE
- Partial indexes and expression indexes
- CREATE INDEX CONCURRENTLY and REINDEX CONCURRENTLY
- Index bloat, write cost and finding unused indexes

Module 8: EXPLAIN ANALYZE, the Planner & Query Tuning (Weeks 4–5)

Reading what the planner decided instead of guessing at it. EXPLAIN (ANALYZE, BUFFERS) line by line — sequential scan against index scan against bitmap heap scan, nested loop against hash join against merge join, and the

estimate-versus-actual row counts that reveal why a plan went wrong.

The planner's inputs are then examined: statistics, ANALYZE, default_statistics_target, extended statistics for correlated columns, and the cost settings — random_page_cost in particular — that are still tuned for spinning disks on many default installations. Work is done on a dataset of over ten million rows, using pg_stat_statements to choose which query to fix first, and practising the rewrites that matter: sargable predicates, keyset pagination instead of a deep OFFSET, and fixing ORM N+1 patterns.

- EXPLAIN (ANALYZE, BUFFERS) read line by line
- Sequential, index, index-only and bitmap heap scans
- Nested loop, hash join and merge join
- Estimate vs actual rows — spotting a bad plan
- Statistics, ANALYZE and extended statistics
- Cost settings, random_page_cost and work_mem
- pg_stat_statements for choosing what to fix
- Sargable predicates and keyset pagination
- Diagnosing and fixing ORM N+1 patterns

Module 9: Transactions, MVCC, VACUUM & Concurrency (Week 5)

How Postgres keeps readers and writers out of each other's way, and what it costs. MVCC explained through row versions, xmin and xmax, and dead tuples; then the four isolation levels demonstrated live in two sessions — Read Committed as the default, Repeatable Read as a true snapshot, and Serializable implemented through Serializable Snapshot Isolation, which fails transactions rather than blocking them and therefore requires retry logic in the application.

VACUUM follows as the direct consequence of MVCC: autovacuum tuning, table and index bloat, freezing and transaction-ID wraparound. Explicit locking, SELECT FOR UPDATE, SKIP LOCKED for queue-as-table workloads, advisory locks and deadlock diagnosis close the module.

- MVCC, row versions, xmin, xmax and dead tuples
- The four isolation levels, demonstrated side by side
- Serializable Snapshot Isolation and retry logic
- VACUUM, autovacuum tuning and bloat
- Freezing and transaction-ID wraparound
- Row and table locks, and lock queues
- SELECT FOR UPDATE and SKIP LOCKED queues
- Advisory locks for cross-session coordination
- Deadlock detection and designing them out

Module 10: Server-Side Programming — PL/pgSQL, Triggers & Views (Week 6)

Logic that belongs in the database, and an honest account of what does not. PL/pgSQL function and procedure syntax, variables, control flow, cursors, exception blocks, and the RETURNS TABLE and set-returning forms; the difference between a function and a PROCEDURE that can manage its own transactions; and SECURITY DEFINER with the search-path trap that turns it into a privilege-escalation hole.

Triggers cover BEFORE, AFTER and INSTEAD OF, statement versus row level, and the audit-capture pattern that is their strongest use. Views, updatable views and materialised views with REFRESH . . . CONCURRENTLY follow, and the module closes on the judgement: business rules generally belong in application code where they can be tested and reviewed, while integrity and audit are the cases the database wins.

- PL/pgSQL functions, procedures and parameters
- Control flow, cursors and RETURNS TABLE
- Exception blocks and RAISE
- SECURITY DEFINER and the search-path trap
- Triggers — BEFORE, AFTER, INSTEAD OF, statement vs row
- Audit capture with triggers
- Views, updatable views and CHECK OPTION

- Materialised views and REFRESH CONCURRENTLY
- Where business logic should actually live

Module 11: Application Integration & Schema Migrations (Week 6)

Connecting Postgres to the code that uses it. Drivers and pooling across the stacks Pune teams run — JDBC with Spring Boot and Hibernate, SQLAlchemy and Django, Prisma and node-postgres, Go's pgx — and pooling explained through what actually happens at pool exhaustion, including why PgBouncer in transaction mode breaks prepared statements and session state.

Parameterised statements are non-negotiable, with SQL injection demonstrated against a vulnerable endpoint and then closed. Migrations are then treated as production discipline: Flyway, Liquibase, Alembic or framework tooling; expand-and-contract so a rename ships without downtime; and the lock-timeout habit that stops a migration from taking an ACCESS EXCLUSIVE lock and stalling every request on a live table.

- JDBC, SQLAlchemy, Django, Prisma, node-postgres and pgx
- Connection pooling, PgBouncer and transaction mode
- Prepared statements, and what pooling breaks
- SQL injection, demonstrated and then closed
- Transaction boundaries in application services
- Migrations with Flyway, Liquibase and Alembic
- Expand-and-contract for zero-downtime changes
- Lock timeouts and migrations that stall production
- LISTEN / NOTIFY and Postgres as a lightweight queue

Module 12: Full-Text Search, pgvector & PostGIS (Week 7)

The extensions that let one database do work teams often add a second system for. Full-text search first — tsvector, tsquery, dictionaries and stemming, ranking with ts_rank, highlighting, and a GIN-indexed search that is fast enough that a small product does not need Elasticsearch.

pgvector follows, and it is the reason many Pune AI features never leave Postgres: embedding columns, cosine and inner-product distance, HNSW and IVFFlat indexes and the recall-versus-speed trade-off between them, and a retrieval-augmented generation query that filters on relational predicates and ranks by vector distance in one statement. PostGIS, TimescaleDB and the foreign-data-wrapper pattern are covered at working depth.

- tsvector, tsquery, dictionaries and stemming
- Ranking, highlighting and GIN-indexed search
- When full-text search beats adding Elasticsearch
- pgvector embedding columns and distance operators
- HNSW vs IVFFlat, and recall against speed
- Hybrid RAG queries — relational filter plus vector rank
- PostGIS for geospatial data and queries
- TimescaleDB for time-series workloads
- Foreign data wrappers for cross-database access

Module 13: Partitioning, Replication & High Availability (Week 7)

Postgres on more than one table and more than one machine. Declarative partitioning by range, list and hash; partition pruning at plan time and at execution time; partition-wise joins; and the maintenance that partitioning actually requires — creating next month's partition before it is needed, and detaching old ones instead of deleting a hundred million rows.

Replication then covers physical streaming replication with hot standbys, replication slots, synchronous versus asynchronous commit and what each guarantees, and logical replication with publications and subscriptions for selective and cross-version copying. Replica lag and the stale reads it produces are demonstrated, and failover, pg_rewrite, Patroni and connection routing complete the picture.

- Declarative partitioning — range, list and hash
- Partition pruning and partition-wise joins

- Partition maintenance and detaching old data
- Physical streaming replication and hot standbys
- Replication slots and WAL retention
- Synchronous vs asynchronous commit
- Logical replication, publications and subscriptions
- Replica lag, stale reads and read routing
- Failover, pg_rewind and Patroni

Module 14: Roles, Row-Level Security & Compliance (Week 8)

Access control in a database whose permission model is genuinely expressive. Roles as both users and groups, the schema and object privilege hierarchy, DEFAULT PRIVILEGES so that tomorrow's tables are not open by accident, and least privilege applied to a real application account rather than the superuser connection string too many projects ship with.

Row-level security is the module's centrepiece: policies for SELECT, INSERT, UPDATE and DELETE, USING against WITH CHECK, and the multi-tenant pattern where a tenant identifier in the session makes cross-tenant leakage impossible in the database rather than merely unlikely in the code. TLS, scram-sha-256, column encryption and the DPDP and GDPR obligations a schema must be able to honour close the week.

- Roles as users and groups; GRANT and REVOKE
- Schema and object privileges; DEFAULT PRIVILEGES
- Least privilege, and never connecting as superuser
- Row-level security policies and USING vs WITH CHECK
- Multi-tenant isolation enforced by the database
- TLS connections and scram-sha-256
- Column encryption and PII handling
- Auditing with pgaudit
- DPDP and GDPR obligations a schema must support

Module 15: Backup, Point-in-Time Recovery & Managed Cloud Postgres (Week 8)

A backup nobody has restored is not a backup. Logical dumps with pg_dump and pg_dumpall and their format options, physical base backups with pg_basebackup, WAL archiving, and continuous archiving with pgBackRest or Barman. Every student performs a real recovery: drop a table, restore to the second before the mistake, and verify the row counts.

Strategy is then designed rather than assumed — full and incremental schedules, retention, encrypted offsite copies, and the recovery point and recovery time objectives that decide all of it. The module ends on managed Postgres, which is how most new Pune workloads ship: RDS and Aurora PostgreSQL, Azure Flexible Server, Cloud SQL, Neon and Supabase — what each takes over, what it costs, and which skills from this course still matter when it does.

- pg_dump and pg_dumpall, and the format options
- pg_basebackup and physical backups
- WAL archiving and continuous archiving
- pgBackRest and Barman
- A point-in-time recovery exercise, verified
- RPO and RTO as the drivers of backup design
- Encrypted offsite copies and retention
- RDS, Aurora, Azure Flexible Server and Cloud SQL
- Neon, Supabase and serverless Postgres

Module 16: Capstone Project & Interview Preparation (Week 9 + placement prep)

The capstone runs the whole course through one system: a schema modelled from a written brief, loaded with millions of rows, queried through reports built on CTEs and window functions, extended with a jsonb document column and a pgvector similarity search, tuned against measured EXPLAIN (ANALYZE, BUFFERS) output, isolated per tenant with row-level security, backed by a tested point-in-time restore, and documented well enough that a stranger can run it.

Interview preparation targets what Pune panels actually ask a Postgres candidate: explain MVCC and why VACUUM exists, describe a plan you fixed and how you measured it, choose between GIN and B-tree for a given predicate, explain

LATERAL, and defend a jsonb column. Resume, LinkedIn and GitHub are rewritten around the capstone, and two mock interviews are run and reviewed.

- Capstone schema, data load and report queries
- A jsonb document column and a pgvector search feature
- A measured before-and-after tuning exercise
- Row-level security for tenant isolation
- A verified point-in-time restore
- README, ER diagram and data dictionary
- Explaining MVCC, VACUUM and isolation under questioning
- Resume, LinkedIn and GitHub rewritten around the capstone
- Two mock interviews with recorded feedback

Module 17: AI-Assisted Development Workflow (Final Week)

The skill every 2026 hiring panel now probes for — building real work with AI in the loop, responsibly. Learn to drive AI assistants (GitHub Copilot, Claude, Cursor, and IDE-native AI) to scaffold and accelerate the tools and stack this course covers, generate tests, explain and refactor unfamiliar code, and cut the boilerplate — while keeping you firmly in control of every decision. Heavy focus on guardrails: reviewing each AI suggestion, spotting hallucinated APIs or wrong answers, and handling licensing and data-privacy concerns. Close with a mini-project that takes a deliverable end-to-end using an AI-assisted workflow, then fold the same tooling into version control and everyday team practice.

- AI assistants — GitHub Copilot, Claude, Cursor, IDE-native AI
- Effective prompting for this course's stack — scaffolding, boilerplate, config
- AI-assisted test generation and coverage
- Explaining, refactoring, and modernising unfamiliar code with AI
- AI-driven review, error detection, and quality checks
- Generating and maintaining documentation with AI
- AI debugging — interpreting errors, logs, and failing output
- Guardrails — reviewing output, avoiding hallucinations, licensing & data privacy
- Team workflow — AI in the editor, in reviews, and in delivery pipelines
- Mini-project — a deliverable built end-to-end with an AI-assisted workflow

Capstone Projects

Production OLTP Schema with JSONB and Partitioning

A complete production-style OLTP schema (e-commerce, fintech, or SaaS — your choice) using Postgres-specific features — JSONB columns where they earn their place, native partitioning on a high-volume table (orders / events / transactions), proper indexes (GIN for JSONB, B-tree for relational), 10M+ rows of synthetic data, plus 20 analytical queries with EXPLAIN ANALYZE optimisation. Outcome: a public GitHub repository with the schema, sample data, and optimisation report.

- Technology: Postgres 16+
- Technology: JSONB + GIN indexes
- Technology: Native partitioning
- Technology: Window functions + lateral joins
- Technology: EXPLAIN ANALYZE optimisation
- Technology: 10M+ row dataset

AI / RAG Service with pgvector

A retrieval-augmented generation service backed by Postgres + pgvector — pick a real domain corpus, embed via OpenAI / sentence-transformers, store in pgvector, hybrid retrieval (BM25 via tsvector + dense via pgvector + reranking), plus a small FastAPI / Express endpoint. Demonstrates the 2026 Postgres differentiator.

- Technology: Postgres 16+ + pgvector
- Technology: OpenAI embeddings or sentence-transformers
- Technology: Full-text search via tsvector

- Technology: Hybrid retrieval
- Technology: FastAPI or Express endpoint

Time-Series or Geo-Spatial App with TimescaleDB / PostGIS

A time-series analytics app using TimescaleDB (IoT sensor data, financial tick data, app metrics) OR a geo-spatial app using PostGIS (location-based search, geo-fenced notifications). Demonstrates extension fluency.

- Technology: Postgres 16+ + TimescaleDB OR PostGIS
- Technology: Continuous aggregates (TimescaleDB)
- Technology: Spatial indexes (PostGIS)
- Technology: Real-time analytics queries

Career Outcomes

PostgreSQL fluency is the new default expectation on Pune product engineering / fintech / SaaS backend roles in 2026 — Indeed Pune lists 700+ openings explicitly requiring Postgres. The biggest employers are Persistent Systems, BMC Software, Bajaj Finserv, BMW TechWorks India, Mastercard Pune Tech Hub, plus the Pune SaaS / fintech scene.

What pulls a Postgres developer / DBA above the median band: depth on JSONB and modern SQL (window functions, lateral joins, CTEs), demonstrable EXPLAIN-plan optimisation experience, one extension specialisation (pgvector / PostGIS / TimescaleDB), plus production-operations literacy (replication, backup, recovery). Our capstone projects are designed exactly around these signals.

Senior Postgres DBA / Engineer bands at the top end are reported as national figures (Pune-specific Indeed pages do not exist for these specific titles); Pune trends within $\pm 10\%$ of these figures.

Roles after this course

- PostgreSQL Developer
- Database Administrator (Postgres)
- Backend Developer (with Postgres depth)
- Analytics Engineer (Postgres + dbt)
- Data Engineer (Postgres-heavy)
- AI Engineer (with pgvector specialisation)

Indicative salary bands

- PostgreSQL Developer / DBA (Pune) - Rs. 6,98,000 per year average (source: Indeed Pune (PostgreSQL Developer))
- Junior Postgres Developer (Pune entry, <2 years) - Rs. 4,00,000 – Rs. 7,00,000 per year (source: AmbitionBox Pune Postgres Developer)
- Mid-level Postgres Developer / DBA (Pune, 3–5 years) - Rs. 10,00,000 – Rs. 17,00,000 per year (source: Glassdoor Pune Postgres DBA)
- Senior Postgres DBA (Pune, 5–8 years) - Rs. 16,00,000 – Rs. 26,00,000 per year (source: Glassdoor Pune Senior Postgres DBA)

Companies hiring for these skills

- Persistent Systems
- BMC Software
- Bajaj Finserv
- BMW TechWorks India
- Mastercard Pune Tech Hub
- Synechron
- Amagi
- Fyllo
- BharatPe (Pune)
- Razorpay (Pune)
- Pine Labs (Pune)
- Drip Capital

- Innovaccer (Pune)
- Whatfix (Pune)
- TCS
- Cognizant

Batch Modes and Duration

Duration: 8 weeks of structured curriculum plus 1 week of capstone (~2 months total)

Classroom

- Location: Archer Infotech, Kothrud, Pune
- Morning batch — 10:00 to 13:00
- Evening batch — 18:00 to 21:00

Live online

- Same hours as classroom batches
- Recordings available for review
- Zoom for live sessions
- psql + DBeaver / pgAdmin / TablePlus
- GitHub for capstone
- Slack / WhatsApp for async Q&A

Weekend

- Saturday + Sunday, 09:00 to 13:00
- Stretches over ~3.5 months instead of 2.

Maximum 15 students per batch.

Fees

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession.

Indicative range: Rs. 20,000 – Rs. 90,000

- Single payment with early-bird discount
- EMI in 2 instalments at no extra cost
- Corporate sponsorship — invoiced with GST

Placement Support

Placement support starts from week 6. By the end of the curriculum your resume highlights real EXPLAIN-plan optimisation work and at least one extension-based project, your GitHub has at least two production-style repositories, and you have completed at least two mock technical interviews.

We say placement support, not placement guarantee. Our support is unconditional, time-bound (six months after course completion), and includes free re-entry to a future batch's interview-prep sessions.

The placement process

- Week 6 — resume + LinkedIn rewrite
- Week 7 — GitHub portfolio cleanup
- Weeks 8–9 — two rounds of mock technical interviews
- Week 9 — HR mock and salary negotiation coaching
- Post-course — referrals via our 17-year alumni network at 12+ partner companies
- Up to 6 months of continued support
- Free re-entry to future batch interview-prep sessions

Partner companies

- Persistent Systems

- BMC Software
- Bajaj Finserv
- BMW TechWorks India
- Mastercard Pune Tech Hub
- Amagi
- Razorpay (Pune)
- TCS
- Cognizant

Prerequisites and How to Start

Prerequisites: basic computer use, willingness to commit 6–8 hours per week of practice. No prior SQL or programming experience required. If you have done our MySQL course or basic SQL, you'll move slightly faster.

Suggested first steps

- Decide your mode — classroom, online live, or weekend
- Check the upcoming batch dates
- Book a free 30-minute counselling call
- Confirm enrolment and complete pre-course orientation (Postgres 16+ install scripts, Neon free-tier signup)
- Show up to day one with a laptop running 64-bit OS

Frequently Asked Questions

How long does PostgreSQL training in Pune take at Archer Infotech?

Approximately 2 months — 8 weeks plus 1 week of capstone. Weekend batch ~3.5 months.

PostgreSQL or MySQL?

Postgres for modern Pune product engineering / fintech / SaaS / analytics. MySQL for LAMP / WordPress / digital-agency / smaller-startup. Postgres has eclipsed MySQL as the default for new Pune product work.

What is the salary of a Postgres Developer / DBA in Pune?

Indeed Pune Rs. 6.98 lakh average. Junior Rs. 4–7 lakh per AmbitionBox. Mid-level Rs. 10–17 lakh per Glassdoor. Senior DBA Rs. 16–26 lakh.

Will I work on real projects?

Yes — three capstone projects: (1) production OLTP schema with JSONB and partitioning, (2) AI / RAG service with pgvector, (3) time-series or geo-spatial app with TimescaleDB / PostGIS.

Is pgvector / RAG covered?

Yes — week 8 covers pgvector and the AI / RAG patterns. Capstone Project #2 is a complete pgvector-backed RAG service. This is the 2026 Postgres differentiator.

Are weekend Postgres classes available in Pune?

Yes — Saturday and Sunday, 09:00–13:00, stretched over ~3.5 months.

What is the fee?

Course fees range Rs. 20,000 – Rs. 90,000 depending on mode and concession.

What support do I get after course completion?

Six months of placement support, referrals via our alumni network, mock interviews.

Is the named trainer actually teaching?

Ankita Hartale personally leads every session.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.

ARCHER INFOTECH PUNE
archerinfotech.in